

CS 229, Winter 2026

Problem Set #1

YOUR NAME HERE (YOUR SUNET HERE)

Due Wednesday, January 21 at 11:59 pm on Gradescope.

Notes: (1) These questions require thought, but do not require long answers. Please be as concise as possible.

(2) If you have a question about this homework, we encourage you to post your question on our Ed forum, at <https://edstem.org/us/courses/67631/discussion/>.

(3) If you missed the first lecture or are unfamiliar with the collaboration or honor code policy, please read the policy on the course website before starting work.

(4) For the coding problems, you may not use any libraries except those defined in the provided `environment.yml` file. In particular, ML-specific libraries such as scikit-learn are not permitted.

(5) The due date is Wednesday, January 21 at 11:59 pm. If you submit after Wednesday, January 21 at 11:59 pm, you will begin consuming your late days. The late day policy can be found in the course website: Course Logistics and FAQ.

All students must submit an electronic PDF version of the written question including plots generated from the codes. We highly recommend typesetting your solutions via L^AT_EX. All students must also submit a zip file of their source code to Gradescope, which should be created using the `make_zip.py` script. You should make sure to (1) restrict yourself to only using libraries included in the `environment.yml` file, and (2) make sure your code runs without errors. Your submission may be evaluated by the auto-grader using a private test set, or used for verifying the outputs reported in the writeup. Please make sure that your PDF file and zip file are submitted to the corresponding Gradescope assignments respectively. We reserve the right to not give any points to the written solutions if the associated code is not submitted.

Honor code: We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solution independently, and without referring to written notes from the joint session. Each student must understand the solution well enough in order to reconstruct it by him/herself. It is an honor code violation to copy, refer to, or look at written or code solutions from a previous year, including but not limited to: official solutions from a previous year, solutions posted online, and solutions you or someone else may have written up in a previous year. Furthermore, it is an honor code violation to post your assignment solutions online, such as on a public git repo. We run plagiarism-detection software on your code against past solutions as well as student submissions from previous years. Please take the time to familiarize yourself with the Stanford Honor Code¹ and the Stanford Honor Code² as it pertains to CS courses.

¹<https://communitystandards.stanford.edu/policies-and-guidance/honor-code>

²<https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1164/handouts/honor-code.pdf>

1. [30 points] **Convergence and Learning Rate for Gradient Descent**

When running an optimization algorithm, finding a good learning rate may require some tuning. If the learning rate is too high, the gradient descent (GD) algorithm might not converge. If the learning rate is too low, GD may take too long to converge. In this question, we will investigate some theoretical guarantees for the convergence of GD on convex objective functions, and their implications on choosing the learning rate.

Recap and notation. Suppose we have a convex objective function $J(\theta)$. At each iteration, GD updates the iterate $\theta^{[t]}$ as follows:

$$\theta^{[t]} = \theta^{[t-1]} - \alpha \nabla J(\theta^{[t-1]})$$

where α is the learning rate and $\nabla J(\theta^{[t-1]})$ is the gradient of $J(\theta)$ evaluated at $\theta^{[t-1]}$.

(a) [8 points] **Quadratic Objective, Scalar Variable**

In this part, consider the simple objective function with one-dimensional variable θ :

$$J(\theta) = \frac{1}{2}\beta\theta^2$$

where $\theta \in \mathbb{R}$ is our parameter and $\beta > 0$ is a positive, constant scalar. We initialize GD at some initialization $\theta^{[0]} \neq 0$ and run GD with learning rate α .

- i. **Derive** the range of α such that the iterate of GD converges in the sense that there exists a scalar $\theta^\dagger$ such that $\lim_{t \rightarrow \infty} |\theta^{[t]} - \theta^\dagger| = 0$. Provide the value of $\theta^\dagger$ when GD converges and show that it's equal to the global minimum $\theta^* = \arg \min_{\theta} J(\theta)$. The range of α can potentially depend on β .
- ii. Given a desired accuracy $\epsilon > 0$, for all the α in the range that you derived, **derive** the minimum number of iterations T required to reach a point $\theta^{[T]}$ such that $|\theta^{[T]} - \theta^*| \leq \epsilon$. T can potentially depend on ϵ , $\theta^{[0]}$, α , and β . Investigate the behavior of T and whether T is increasing or decreasing for different values of α . Briefly discuss why choosing an inappropriate α can cause the algorithm to take longer to converge.

Hint: Express $\theta^{[t]}$ in terms of $\theta^{[0]}$, α , t , and β .

Hint: $\lim_{t \rightarrow \infty} c^t = 0$, $\forall c \in \mathbb{R} \text{ s.t. } |c| < 1$

Answer:

(b) [4 points] **Quadratic Objective, d-dimensional Variable**

In this part of the question, consider the objective function with d -dimensional variable θ :

$$J(\theta) = \frac{1}{2} \sum_{i=1}^d \beta_i \theta_i^2$$

where θ_i 's are our parameters and $\beta_i \in \mathbb{R} \text{ s.t. } \beta_i > 0$ are positive, constant scalars. Assume that we start from some $\theta^{[0]}$ where $\theta_i^{[0]} \neq 0$ for all i and run GD with learning rate α . Derive the range of learning rate α such that GD converges in the sense that there exists a vector $\theta^\dagger \in \mathbb{R}^d$ such that $\lim_{t \rightarrow \infty} \|\theta^{[t]} - \theta^\dagger\|_2 = 0$. Provide the value of $\theta^\dagger$ when GD converges. The range of α can potentially depend on β_i where $i \in \{1, \dots, d\}$.

Answer:

(c) [4 points] **Coding Question: Quadratic Multivariate Objective**

Let the objective function be

$$J(\theta) = \theta^\top A \theta$$

where $A \in \mathbb{R}^{2 \times 2}$ is a 2×2 real, positive semi-definite matrix. Do the following exercises:

- i. Implement the `update_theta` and `gradient_descent` function in `src/gd_convergence/experiment.py`. You can stop the GD algorithm when either of the following condition is satisfied:

- A. $|J(\theta^{[t]}) - J(\theta^{[t-1]})| < \epsilon$ where $\epsilon = 10^{-50}$ is given as the function parameter. This is when we assume the algorithm converged.
B. $J(\theta^{[t]}) > 10^{20}$. This is to prevent an infinite loop when the algorithm does not converge.

To test your implementation, run `python src/gd_convergence/experiment.py`, which checks that your θ (approximately) converges to the optimal value.

Note that we have provided you a matrix $A = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$ and $\theta^{[0]} = [-1, 0.5]$ at the beginning of the file for you to experiment with. Therefore, the objective function is a special case of the objective function in part (b) with dimension $d = 2$. Check if your theoretical derivation of the feasible range of the learning rate indeed matches empirical observations.

- ii. Now, suppose we rotate the matrix A , what do we observe? Plot the trajectories of the GD algorithm using the following learning rates: 0.05, 0.1, 0.2, 0.3, 0.4, 0.45, 0.5, and 1. We have provided the rotation and plotting function for you. You need to simply run `python src/gd_convergence/plotting.py lr1 lr2 lr3 ...` with `lr*` replaced with desired learning rates. Include the output file `trajectories.png` and `trajectories_rotated.png`, as well as a brief discussion on the printed output of `plotting.py` in the write-up.

Remark: Setting the learning rate too high will cause the objective function to not converge. The convergence properties of these learning rates are also rotational invariant. We will show this formally in the next part of the problem.

Answer:

(d) [12 points] **Convergence of Gradient Descent for a General Convex Objective**

Now let's consider any convex, twice continuously differentiable objective function $J(\theta)$ that is bounded from below. Suppose that the largest eigenvalue of the Hessian matrix $H = \nabla^2 J(\theta)$ is less than or equal to $\beta_{\max}$ for all points θ .

Show that when running gradient descent, choosing a step size $0 < \alpha < \frac{1}{\beta_{\max}}$ guarantees that $J(\theta^{[t]})$ will converge to some finite value as t approaches infinity.³ Specifically, you should derive an inequality for $J(\theta^{[t]})$ in terms of $J(\theta^{[t-1]})$, $\nabla J(\theta^{[t-1]})$, α , and $\beta_{\max}$, and show that the objective function is strictly decreasing during each iteration, i.e. $J(\theta^{[t]}) < J(\theta^{[t-1]})$.

Hint: You can assume that, by Taylor's theorem, the following statement is true:

$$J(y) = J(x) + \nabla J(x)^\top (y - x) + \frac{1}{2} (y - x)^\top \nabla^2 J(x + c(y - x)) (y - x) \text{ for some } 0 \leq c \leq 1$$

³This definition of convergence is different from the definition in previous questions where we explicitly prove that the parameter θ converges to an optimal parameter θ^* . In this case, we do not know the optimal value θ^* , and it would be difficult to prove convergence using the previous definition.

Hint: The Hessian of a convex function is symmetric and positive semi-definite at any point θ , which means all of its eigenvalues are real and non-negative.

Hint: The Hessian matrix is symmetric. Consider using the spectral theorem introduced in problem 3 in homework 0.

Optional (no credit): Using the gradient descent inequality that you derived, show that the GD algorithm converges in the sense that $\lim_{t \rightarrow \infty} \|\nabla J(\theta^{[t]})\|_2^2 = 0$.⁴

Remark: This question suggests that a smaller learning rate should be applied if we believe the curvature of the objective function is big.

Answer:

(e) **[2 points] Learning Rate for Linear Regression**

Consider using GD on the LMS objective introduced in lecture:

$$J(\theta) = \frac{1}{2} \|X\theta - y\|_2^2$$

where $X \in \mathbb{R}^{n \times d}$ is the design matrix of our data, $\theta \in \mathbb{R}^d$ is our parameter, and $\vec{y} \in \mathbb{R}^n$ is the response variable.

Let $\beta_{\max}$ be the largest eigenvalue of $X^\top X$. Prove that for all $\alpha \in (0, \frac{1}{\beta_{\max}})$, GD with learning rate α satisfies that $J(\theta^{[t]})$ converges as $t \rightarrow \infty$.

Hint: You can invoke the statements in the part (d) (including the optional question in part (d)) to solve this part (even if you were not able to prove part (d)).

Remark: The conclusion here suggests that for linear regression, roughly speaking, you should use smaller learning rates if the scale of your data X is big or if the data points are correlated with each other, both of which will enable a large top eigenvalue of $XX^\top$.

Remark: Even though in many cases the LMS objective can be solved exactly by solving the normal equation as shown in lecture, it is still useful to be able to guarantee convergence when using the Gradient Descent algorithm in situations where it is difficult to solve for the optimal θ^* directly (such as having large amount of data making inverting X very expensive).

Answer:

⁴Note that $\lim_{t \rightarrow \infty} \|\nabla J(\theta^{[t]})\|_2^2 = 0$ does not necessarily mean that there exists a vector $\hat{\theta}$ such that $\theta^{[t]} \rightarrow \hat{\theta}$. (Even an 1-dimensional counterexample exists.) However, for convex functions, when $\theta^{[t]}$ stays in a bounded set, $\lim_{t \rightarrow \infty} \|\nabla J(\theta^{[t]})\|_2^2 = 0$ does imply that $J(\theta^{[t]}) \rightarrow \inf_{\theta} J(\theta)$ as $t \rightarrow \infty$. Therefore, $\lim_{t \rightarrow \infty} \|\nabla J(\theta^{[t]})\|_2^2 = 0$ is typically considered a reasonable definition for an optimization algorithm's convergence.

2. [15 points] Linear Classification (logistic regression)

In this problem, we cover the linear classification task using logistic regression. The goal is to find a linear decision boundary that separates the data into two classes. We will consider two datasets, along with starter codes provided in the following files:

- `src/linearclass/ds1_{train,valid}.csv`
- `src/linearclass/ds2_{train,valid}.csv`
- `src/linearclass/logreg.py`

Each file contains n examples, one example $(x^{(i)}, y^{(i)})$ per row. In particular, the i -th row contains columns $x_1^{(i)} \in \mathbb{R}$, $x_2^{(i)} \in \mathbb{R}$, and $y^{(i)} \in \{0, 1\}$.

Typically, a trained model is evaluated by its performance on the validation dataset. The validation dataset is a set of examples drawn from the same (or a similar) distribution as the training data. Intuitively, this is because we need the trained model to correctly predict the label for not only the training data, but also new samples from the same distribution.

(a) [5 points]

In lecture we saw the average empirical loss for logistic regression:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n \left(y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right),$$

where $y^{(i)} \in \{0, 1\}$, $h_{\theta}(x) = g(\theta^T x)$ and $g(z) = 1/(1 + e^{-z})$.

Find the Hessian H of this function, and show that for any vector z , it holds true that

$$z^T H z \geq 0.$$

Hint: You may want to start by showing that $\sum_i \sum_j z_i x_i x_j z_j = (x^T z)^2 \geq 0$. Recall also that $g'(z) = g(z)(1 - g(z))$.

Remark: This is one of the standard ways of showing that the matrix H is positive semi-definite, written “ $H \succeq 0$.” This implies that J is convex, and has no local minima other than the global one. If you have some other way of showing $H \succeq 0$, you’re also welcome to use your method instead of the one above.

Answer:

(b) [10 points] **Coding problem.** In `src/linearclass/logreg.py`, write the code to train a logistic regression classifier using Newton’s Method. Starting with $\theta = \vec{0}$, run Newton’s Method until the updates to θ are small: Specifically, train until the first iteration k such that $\|\theta_k - \theta_{k-1}\|_1 < \epsilon$, where $\epsilon = 1 \times 10^{-5}$. Make sure to write the model’s predicted probabilities on the validation set to the file specified in the code.

Include a plot of the **validation data** with x_1 on the horizontal axis and x_2 on the vertical axis. To visualize the two classes, use a different symbol for examples $x^{(i)}$ with $y^{(i)} = 0$ than for those with $y^{(i)} = 1$. On the same figure, plot the decision boundary found by logistic regression (i.e., line corresponding to $p(y|x) = 0.5$). **There is a plotting function provided in util.py.**

Note: If you want to print the loss during training, you may encounter some numerical instability issues. Recall that the loss function on an example (x, y) is defined as

$$y \log(h_{\theta}(x)) + (1 - y) \log(1 - h_{\theta}(x)),$$

where $h_\theta(x) = (1 + \exp(-x^\top \theta))^{-1}$. Technically speaking, $h_\theta(x) \in (0, 1)$ for any $\theta, x \in \mathbb{R}^d$. However, in Python a real number only has finite precision. So it is possible that in your implementation, $h_\theta(x) = 0$ or $h_\theta(x) = 1$, which makes the loss function ill-defined. A typical solution to the numerical instability issue is to add a small perturbation. In this case, you can compute the loss function using

$$y \log(h_\theta(x) + \epsilon) + (1 - y) \log(1 - h_\theta(x) + \epsilon),$$

instead, where ϵ is a very small perturbation (for example, $\epsilon = 10^{-5}$). **Answer:**

3. [25 points] Learning from imbalanced datasets

In this problem, we study how to learn a classifier from an imbalanced dataset, where the marginal distribution of the classes/labels are imbalanced. Imbalanced datasets are ubiquitous in real-world applications. For example, in the spam detection problem, the training dataset usually has only a small fraction of spam emails (positive examples) but a large fraction of ordinary emails (negative examples). For simplicity, we consider the binary classification problem where the labels are in $\{0, 1\}$ and the number of positive examples is much smaller than the number of negative examples.

(a) [10 points] Evaluation metrics

In this sub-question, we discuss the common evaluation metrics for imbalanced datasets. Suppose for some $\rho \in (0, 1)$, we assume that a fraction ρ of the examples in the validation dataset are positive examples (with label 1), and a fraction $1 - \rho$ of them are negative examples (with label 0).

Define the accuracy as

$$A \triangleq \frac{\# \text{examples that are predicted correctly by the classifier}}{\# \text{examples}}$$

(i) (3 point) Show that for any dataset with ρ fraction of positive examples and $1 - \rho$ fraction of negative examples, there exists a (trivial) classifier with accuracy at least $1 - \rho$.

The statement above suggests that the accuracy is not an ideal evaluation metric when ρ is close to 0. E.g., imagine that for spam detection ρ can be smaller than 1%. The statement suggests there is a trivial classifier that gets more than 99% accuracy. This could be misleading — 99% seems to be almost perfect, but actually you don't need to learn anything from the dataset to achieve it.

Therefore, for imbalanced datasets, we need more informative evaluation metrics. We define the number of true positive, true negative, false positive, false negative examples as

$TP \triangleq \#$ positive examples with a correct (positive) prediction

$TN \triangleq \#$ negative examples with a correct (negative) prediction

$FP \triangleq \#$ negative examples with a incorrect (positive) prediction

$FN \triangleq \#$ positive examples with a incorrect (negative) prediction

Define the accuracy over positive examples as

$$A_1 \triangleq \frac{TP}{TP + FN} = \frac{\# \text{ positive examples with a correct (positive) prediction}}{\# \text{ positive examples}}$$

Define the accuracy of negative examples as

$$A_0 \triangleq \frac{TN}{TN + FP} = \frac{\# \text{ negative examples with a correct (negative) prediction}}{\# \text{ negative examples}}$$

We define the balanced accuracy as

$$\bar{A} \triangleq \frac{1}{2} (A_0 + A_1) \tag{1}$$

With these notations, we can verify that the accuracy is equal to $A = \frac{TP+TN}{TP+TN+FP+FN}$.
(ii) (4 point) Show that

$$\rho = \frac{TP + FN}{TP + TN + FP + FN}$$

and

$$A = \rho \cdot A_1 + (1 - \rho)A_0 \quad (2)$$

Comparing equation (1) and (2), we can see that the accuracy and balanced accuracy are both linear combination of A_0 and A_1 but with different weighting.

(iii) (3 point) Show that the trivial classifier you constructed for part (i) has balanced accuracy 50%.

Partly because of (iii), the balanced accuracy $\bar{A}$ is often a preferable evaluation metric than the accuracy A . Sometimes people also report the accuracies for the two classes (A_0 and A_1) to demonstrate the performance for each class.

Answer:

(b) [5 points] **Coding problem: vanilla logistic regression**

First, we use the vanilla logistic regression to learn an imbalanced dataset. For the rest of the question, we will use the dataset and starter code provided in the following files:

- `src/imbalanced/{train,validation}.csv`
- `src/imbalanced/imbalanced.py`

Each file contains n examples, one example $(x^{(i)}, y^{(i)})$ per row. x is two-dimensional, i.e., the i -th row contains columns $x_1^{(i)} \in \mathbb{R}$, $x_2^{(i)} \in \mathbb{R}$, and $y^{(i)} \in \{0, 1\}$. Let $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ be our training dataset. $\mathcal{D}$ has ρn examples with label 1 and $(1 - \rho)n$ with label 0. In the dataset we constructed, $\rho = 1/11$.

You will train a linear classifier $h_\theta(x)$ with average empirical loss for logistic regression, where $h_\theta(x) = g(\theta^T x)$, $g(z) = 1/(1 + e^{-z})$, similar to Problem 1 Part a:

$$J(\theta) = -\frac{1}{n} \sum_{i=1}^n \left(y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right),$$

You are encouraged to use your code for problem 3, but you are also allowed to use any standard logistic regression library or package to optimize the objective above. After obtaining the classifier, compute the classifier's accuracy (A), balanced accuracy ($\bar{A}$), accuracies for the two classes (A_0, A_1) on the validation dataset, and report them in the writeup. You are expected to observe that the minority class (positive class) has significantly lower accuracy than the majority class.

Create a plot to visualize the validation set with x_1 on the horizontal axis and x_2 on the vertical axis. Use different symbols for examples $x^{(i)}$ with true label $y^{(i)} = 1$ than those with $y^{(i)} = 0$. On the same figure, plot the decision boundary obtained by your model (i.e, line corresponding to model's predicted probability = 0.5) in red color. Include this plot in your writeup. **There is a plotting function provided in util.py.**

Deliverables include:

- the plot

- the classifier's accuracy (A) on the validation dataset
- the balanced accuracy ($\bar{A}$) on the validation dataset
- the accuracies for the two classes (A_0, A_1) on the validation dataset

Answer:

(c) [5 points] **Re-sampling/Re-weighting Logistic Regression**

The relatively low accuracy for the minority class and the resulting low balanced accuracy are undesirable for many applications. Various methods have been proposed to improve the accuracy for the minority class, and learning imbalanced datasets is an active open research direction. Here we introduce a simple and classical re-sampling/re-weighting technique that helps improve the balanced accuracy in most of the cases.

We observe that the logistic regression algorithm works well for the accuracy but not for the balanced accuracy. Let $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ denote the existing training dataset. We will create a new dataset $\mathcal{D}'$ such that the accuracy on $\mathcal{D}'$ is the same as the balanced accuracy on $\mathcal{D}$.

Assume $\rho < 1/2$ without loss of generality, and let $\kappa = \frac{\rho}{1-\rho}$. This means the number of positive examples is κ times the number of negative examples in $\mathcal{D}$. Assume for convenience $1/\kappa$ is an integer.⁵ Let $\mathcal{D}'$ be the dataset that contain each negative example once and $1/\kappa$ repetitions of each positive example in $\mathcal{D}$. Then we will apply logistic regression on the new dataset $\mathcal{D}'$.

Prove that for any classifier, the balanced accuracy on $\mathcal{D}$ is equal to the accuracy on $\mathcal{D}'$. Moreover, **show that** the average empirical loss for logistic regression on the dataset $\mathcal{D}'$ is equal to

$$J(\theta) = -\frac{1+\kappa}{2n} \sum_{i=1}^n w^{(i)} \left(y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right),$$

where $w^{(i)} = 1$ if $y^{(i)} = 0$ and $w^{(i)} = 1/\kappa$ if $y^{(i)} = 1$.

Observe effectively we are re-weighting the loss function for each example by some constant that depends on the frequency of the class it belongs to.

Answer:

(d) [5 points] **Coding problem: re-weighting minority class**

In `src/imbalanced/imbalanced.py`, implement the logistic regression algorithm on $\mathcal{D}'$. Compute and report the classifier's accuracy (A), balanced accuracy ($\bar{A}$), accuracies for the two classes (A_0, A_1) on the validation dataset. You are expected to see that the accuracy of the minority class (class 1) improves significantly, whereas that of the majority class drops compared to vanilla logistic regression. However, the balanced accuracy is significantly greater than that of vanilla logistic regression.

Create a plot to visualize the validation set with x_1 on the horizontal axis and x_2 on the vertical axis. Use different symbols for examples $x^{(i)}$ with true label $y^{(i)} = 1$ than those with $y^{(i)} = 0$. On the same figure, plot the decision boundary obtained by your model (i.e., line corresponding to model's predicted probability = 0.5) in red color. Include this plot in your writeup. **There is a plotting function provided in `util.py`.**

Deliverables include:

- the plot

⁵otherwise we can round up to the nearest integer with introducing a slight approximation.

- the classifier's accuracy (A) on the validation dataset
- the balanced accuracy ($\bar{A}$) on the validation dataset
- the accuracies for the two classes (A_0, A_1) on the validation dataset

Answer:

4. [12 points] Double Descent on Linear Models

Background: In this question, you will empirically observe the sample-wise double descent phenomenon. That is, the validation losses of some learning algorithms or estimators do not monotonically decrease as we have more training examples, but instead have a curve with two U-shaped parts. The double descent phenomenon can be observed even for simple linear models. In this question, we consider the following setup. Let $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$ be the training dataset. Let $X \in \mathbb{R}^{n \times d}$ be the matrix representing the inputs (i.e., the i -th row of X corresponds to $x^{(i)}$), and $\vec{y} \in \mathbb{R}^n$ the vector representing the labels (i.e., the i -th row of $\vec{y}$ corresponds to $y^{(i)}$):

$$X = \begin{bmatrix} - & x^{(1)} & - \\ - & x^{(2)} & - \\ \vdots & \vdots & \vdots \\ - & x^{(n)} & - \end{bmatrix}, \quad \vec{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}.$$

Similarly, we use $X_v \in \mathbb{R}^{m \times d}$, $\vec{y}_v \in \mathbb{R}^m$ to represent the validation dataset, where m is the size of the validation dataset. We assume that the data are generated with $d = 500$.

In this question, we consider *regularized* linear regression. For a regularization level $\lambda \geq 0$, define the regularized cost function

$$J_\lambda(\beta) = \frac{1}{2} \|X\beta - \vec{y}\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2,$$

and its minimizer $\hat{\beta}_\lambda = \arg \min_{\beta \in \mathbb{R}^d} J_\lambda(\beta)$.

- (a) [2 points] In this sub-question, we derive the closed-form solution of $\hat{\beta}_\lambda$. **Prove** that when $\lambda > 0$,

$$\hat{\beta}_\lambda = (X^\top X + \lambda I_{d \times d})^{-1} X^\top \vec{y} \quad (3)$$

(recall that $I_{d \times d} \in \mathbb{R}^{d \times d}$ is the identity matrix.)

Note: $\lambda = 0$ is a special case here. When $\lambda = 0$, $(X^\top X + \lambda I_{d \times d})$ could be singular. Therefore, there might be more than one solutions that minimize $J_0(\beta)$. In this case, we define $\hat{\beta}_0$ in the following way:

$$\hat{\beta}_0 = (X^\top X)^+ X^\top \vec{y}. \quad (4)$$

where $(X^\top X)^+$ denotes the Moore-Penrose pseudo-inverse of $X^\top X$. You don't need to prove the case when $\lambda = 0$, but this definition is useful in the following sub-questions.

Answer:

- (b) [5 points] **Coding question: the double descent phenomenon for unregularized models**

In this sub-question, you will empirically observe the double descent phenomenon. You are given 13 training datasets of sample sizes $n = 200, 250, \dots, 750$, and 800, and a validation dataset, located at

- `src/doubledescent/train200.csv`, `train250.csv`, etc.
- `src/doubledescent/validation.csv`

For each training dataset $(X, \vec{y})$, compute the corresponding $\hat{\beta}_0$, and evaluate the mean squared error (MSE) of $\hat{\beta}_0$ on the validation dataset. The MSE for your estimators $\hat{\beta}$ on a validation dataset $(X_v, \vec{y}_v)$ of size m is defined as:

$$\text{MSE}(\hat{\beta}) = \frac{1}{2m} \|X_v \hat{\beta} - \vec{y}_v\|_2^2.$$

Complete the `regression` method of `src/doubledescent/doubledescent.py` which takes in a training file and a validation file, and computes $\hat{\beta}_0$. You can use `numpy.linalg.pinv` to compute the pseudo-inverse.

In your writeup, include a line plot of the validation losses. The x-axis is the size of the training dataset (from 200 to 800); the y-axis is the MSE on the validation dataset. You should observe that the validation error increases and then decreases as we increase the sample size.

Note: When $n \approx d$, the test MSE could be very large. For better visualization, it is okay if the test MSE goes out of scope in the plot for some points.

Answer:

- (c) [5 points] **Coding question: double descent phenomenon and the effect of regularization.**

In this sub-question, we will show that regularization mitigates the double descent phenomenon for linear regression. We will use the same datasets as specified in sub-question (b). Now consider using various regularization strengths. For $\lambda \in \{0, 1, 5, 10, 50, 250, 500, 1000\}$, you will compute the minimizer of $J_\lambda(\beta)$.

Complete the `ridge_regression` method of `src/doubledescent/doubledescent.py` which takes in a training file and a validation file, computes the $\hat{\beta}_\lambda$ that minimizes the training objective under different regularization strengths, and returns a list of validation errors (one for each choice of λ).

In your writeup, include a plot of the validation losses of these models. The x-axis is the size of the training dataset (from 200 to 800); the y-axis is the MSE on the validation dataset. Draw one line for each choice of λ connecting the validation errors across different training dataset sizes. Therefore, the plot should contain 8×13 points and 8 lines connecting them. You should observe that for some small λ 's, the validation error may increase and then decrease as we increase the sample size. However, double descent does not occur for a relatively large λ .

Remark: If you want to learn more about the double descent phenomenon and the effect of regularization, you can start with this paper Nakkiran, et al. 2020.

Answer: